FinShark

Complete Development Sequence

React + TypeScript frontend — feature-by-feature build log

Feature 1 — Card (first version)

Goal: Get one component on screen showing some company info.

Created src/Components/Card/Card.tsx

src/Components/Card/Card.tsx

```
import React, { JSX } from 'react'
import "./Card.css"

interface Props {
  companyName: string;
  ticker: string;
  price: number;
}

const Card: React.FC<Props> = ({companyName, ticker, price}: Props): JSX.Element => {
  return (
    <div className="card">
      <img src="" alt="" />
      <div className='details'>
        <h2>{companyName} ({ticker})</h2>
        <p>${price}</p>
      </div>
    </div>
  )
}

export default Card
```

Why:

- `interface Props` — declares what props are needed. No interface = no type safety on props.
- `React.FC<Props>` — typed functional component.
- Three primitive props for now. Will be replaced once API data exists.

Feature 2 — CardList v1 (hardcoded)

Goal: Render multiple Cards. No API yet → hardcode.

Created src/Components/CardList/CardList.tsx

src/Components/CardList/CardList.tsx

```
import React, { JSX } from 'react'
import Card from '../Card/Card'

interface Props {}

const CardList: React.FC<Props> = (props: Props): JSX.Element => {
  return (
    <div>
      <Card companyName='Apple' ticker='AAPL' price={100} />
      <Card companyName='Microsoft' ticker='MSFT' price={200}/>
      <Card companyName='Tesla' ticker='TSLA' price={300}/>
    </div>
  )
}

export default CardList;
```

Why:

- Empty Props — placeholder until real data arrives.
- Hardcoded children just to see something rendered.

Updated App.tsx

App.tsx

```
function App() {
  return (
    <div className="App">
      <CardList />
    </div>
  );
}
```

✅ *Three cards on screen.*

Feature 3 — Search component v1 (with button)

Goal: Input box. Lift state to App so other components can read it later.

Created src/Components/Search/Search.tsx

src/Components/Search/Search.tsx

```
import React, { ChangeEvent, JSX, SyntheticEvent } from 'react'

interface Props {
  onClick: (e: SyntheticEvent) => void;
  search: string | undefined;
  handleChange: (e: ChangeEvent<HTMLInputElement>) => void;
}

const Search: React.FC<Props> = ({onClick, search, handleChange}: Props): JSX.Element => {
  return (
    <div>
      <input value={search} onChange={(e) => handleChange(e)}></input>
      <button onClick={(e) => onClick(e)} />
    </div>
  )
}

export default Search;
```

Why:

- Search owns nothing — receives everything from App.
- `value=` + `onChange=` together → controlled input. Without onChange, input looks frozen.
- `ChangeEvent<HTMLInputElement>` — typed event so e.target.value is known to TS.
- Note: separate input + button. Will be refactored away in Feature 5.

Updated App.tsx

App.tsx

```
import { ChangeEvent, SyntheticEvent, useState } from 'react';
import Search from './Components/Search/Search';

function App() {
  const [search, setSearch] = useState<string>("");

  const handleChange = (e: ChangeEvent<HTMLInputElement>) => {
    setSearch(e.target.value);
  }

  const onClick = (e: SyntheticEvent) => {
    console.log(search);
  }

  return (
    <div className="App">
      <Search onClick={onClick} search={search} handleChange={handleChange}/>
      <CardList />
    </div>
  );
}
```

Why:

- `useState<string>("")` — typed state, empty initial.
- Handlers defined in App, drilled down as props.
- `onClick` just logs for now — API call comes next.

✅ *Type, see value reflected. Click → console logs the search.*

Feature 4 — API call

Goal: Click search → fetch from FMP → log results.

4.1 — Install axios

```
npm install axios
```

4.2 — Created src/company.d.ts

src/company.d.ts

```
export interface CompanySearch {
  currency: string;
  exchangeShortName: string;
  name: string;
  stockExchange: string;
  symbol: string;
}
```

Why:

- `.d.ts` = type-only file. No runtime code.
- Matches the shape FMP returns for search results.

4.3 — Created .env in project root

```
REACT_APP_API_KEY=your_key_here
```

Why:

- Key stays out of source. Add `.env` to `.gitignore`.
- CRA only exposes vars prefixed `REACT_APP_`.

4.4 — Created src/api.tsx

src/api.tsx

```
import axios from "axios"
import { CompanySearch } from "./company"

interface SearchResponse {
  data: CompanySearch[];
}

export const searchCompanies = async (query: string) => {
  try {
    const data = await axios.get<SearchResponse>(
      `https://financialmodelingprep.com/stable/search-
symbol?query=${query}&limit=10&exchange=NASDAQ&apikey=${process.env.REACT_APP_API_KEY}`
    );
    return data;
  }
  catch (error) {
    if (axios.isAxiosError(error)) {
      console.log("Axios error message: ", error.message);
      return error.message;
```

```
    }
    else {
      console.log("unexpected error: ", error);
      return "An unexpected error has occured";
    }
  }
}
```

Why:

- `async / await` — read top to bottom, no .then() chains.
- `axios.get<SearchResponse>(...)` — generic types the response.
- Backtick template literal for `${query}` and `${process.env.REACT_APP_API_KEY}`.
- `try/catch` returns either response object (success) or string (error). Caller has to handle both.
- `axios.isAxiosError(error)` — type guard, narrows error so error.message is readable.

4.5 — Updated App.tsx

App.tsx

```
const [search, setSearch] = useState<string>("");
const [searchResult, setSearchResult] = useState<CompanySearch[]>([]);
const [serverError, setServerError] = useState<string>("");

const onClick = async (e: SyntheticEvent) => {
  const result = await searchCompanies(search);
  if (typeof result === "string") {
    setServerError(result);
  }
  else if (Array.isArray(result.data)) {
    setSearchResult(result.data);
  }
}

return (
  <div className="App">
    <Search onClick={onClick} search={search} handleChange={handleChange}/>
    {serverError && <h1>{serverError}</h1>}
    <CardList />
  </div>
);
```

Why:

- New state slots: `searchResult` (success), `serverError` (failure).
- `onClick` is now `async` because it awaits the API.
- `typeof result === "string"` catches the error case. `Array.isArray(...)` confirms data exists.
- `{serverError && <h1>...}` — empty string is falsy → renders nothing. With content → h1 shows.

✅ *Type "tsla", click button, console logs the array.*

Feature 5 — Map results into Cards (CardList v2) + Portfolio

Multiple files change in dependency order.

5.1 — Updated Card.tsx — full search result instead of three primitives

src/Components/Card/Card.tsx

```
import React, { JSX, SyntheticEvent } from 'react'
import "./Card.css"
import { CompanySearch } from '../../company'
import AddPortfolio from '../Portfolio/AddPortfolio/AddPortfolio'

interface Props {
  id: string;
  searchResult: CompanySearch;
  onPortfolioCreate: (e: SyntheticEvent) => void;
}

const Card: React.FC<Props> = ({id, searchResult, onPortfolioCreate}: Props): JSX.Element => {
  return (
    <div className="card" key={id}>
      <img src="" alt="" />
      <div className='details'>
        <h2>{searchResult.name} ({searchResult.symbol})</h2>
        <p>{searchResult.currency}</p>
      </div>
      <AddPortfolio onPortfolioCreate={onPortfolioCreate} symbol={searchResult.symbol}/>
    </div>
  )
}

export default Card
```

Why:

- One `searchResult: CompanySearch` instead of three primitives — matches API shape.
- Renders `<AddPortfolio />` inside, drilling onPortfolioCreate further.

5.2 — Created src/Components/Portfolio/AddPortfolio/AddPortfolio.tsx

src/Components/Portfolio/AddPortfolio/AddPortfolio.tsx

```
import React, { SyntheticEvent } from 'react'

interface Props {
  symbol: string;
  onPortfolioCreate: (e: SyntheticEvent) => void;
}

const AddPortfolio = ({onPortfolioCreate, symbol}: Props) => {
  return (
    <form onSubmit={(e) => onPortfolioCreate(e)}>
      <input readOnly={true} hidden={true} value={symbol}/>
      <button type="submit">Add</button>
    </form>
  )
}
```

```
export default AddPortfolio
```

Why:

- Hidden input carries symbol through the form. On submit, `e.target[0].value` reads it back.
- `<form onSubmit>` — works for both Enter key and clicks.

5.3 — Installed uuid

```
npm install uuid
npm install @types/uuid
```

5.4 — Updated CardList.tsx (v2 — maps real data)

src/Components/CardList/CardList.tsx

```
import React, { SyntheticEvent } from 'react'
import { CompanySearch } from '../../company'
import Card from '../Card/Card'
import { v4 as uuidv4 } from "uuid"

interface Props {
  searchResults: CompanySearch[];
  onPortfolioCreate: (e: SyntheticEvent) => void;
}

const CardList: React.FC<Props> = ({searchResults, onPortfolioCreate}: Props): JSX.Element => {
  return (
    <>
      {searchResults.length > 0 ? (
        searchResults.map((result) => {
          return (
            <Card
              id={result.symbol}
              key={uuidv4()}
              searchResult={result}
              onPortfolioCreate={onPortfolioCreate}
            />
          );
        })
      ) : (
        <h1>No results.</h1>
      )}
    </>
  );
}

export default CardList;
```

Why:

- Empty Props replaced with `searchResults` array + `onPortfolioCreate` handler.
- `searchResults.map(...)` renders one Card per result.
- Empty-state ternary fallback.
- `key={uuidv4()}` — unique key per child. Stable id from data (`result.symbol`) would be even better.

5.5 — Updated App.tsx — portfolio state, create/delete handlers, refactored Search

App.tsx

```
const [search, setSearch] = useState<string>("");
const [portfolioValues, setPortfolioValues] = useState<string[]>([]);
const [searchResult, setSearchResult] = useState<CompanySearch[]>([]);
const [serverError, setServerError] = useState<string>("");

const handleSearchChange = (e: ChangeEvent<HTMLInputElement>) => {
  setSearch(e.target.value);
}

const onPortfolioCreate = (e: any) => {
  e.preventDefault();
  const exists = portfolioValues.find((value) => value === e.target[0].value);
  if (exists) return;
  const updatedPortfolio = [...portfolioValues, e.target[0].value];
  setPortfolioValues(updatedPortfolio);
}

const onPortfolioDelete = (e: any) => {
  e.preventDefault();
  const removed = portfolioValues.filter((value) => value !== e.target[0].value);
  setPortfolioValues(removed);
}

const onSearchSubmit = async (e: SyntheticEvent) => {
  e.preventDefault();
  const result = await searchCompanies(search);
  if (typeof result === "string") {
    setServerError(result);
  }
  else if (Array.isArray(result.data)) {
    setSearchResult(result.data);
  }
}

return (
  <div className="App">
    <Search onSearchSubmit={onSearchSubmit} search={search}
handleSearchChange={handleSearchChange}/>
    <ListPortfolio portfolioValues={portfolioValues} onPortfolioDelete={onPortfolioDelete}/>
    <CardList searchResults={searchResult} onPortfolioCreate={onPortfolioCreate}/>
    {serverError && <h1>{serverError}</h1>}
  </div>
);
```

Why:

- New state: `portfolioValues: string[]` — array of symbols user added.
- Renamed handlers (`handleChange` → `handleSearchChange`, `onClick` → `onSearchSubmit`) — clearer with multiple forms now.
- **`onPortfolioCreate`**:
- • `preventDefault()` — stop form's default page reload.
- • `.find()` — duplicate check. Returns the value (truthy) or undefined (falsy). Truthy → return early.
- • `[...portfolioValues, newValue]` — spread → new array → new reference → React re-renders. Never .push().
- **`onPortfolioDelete`**:
- • `.filter()` — returns new array without the matched item. Doesn't mutate.

5.6 — Refactored Search.tsx — button removed, form takes over

src/Components/Search/Search.tsx

```
interface Props {
  onSearchSubmit: (e: SyntheticEvent) => void;
  search: string | undefined;
  handleSearchChange: (e: ChangeEvent<HTMLInputElement>) => void;
}

const Search = ({onSearchSubmit, search, handleSearchChange}: Props) => {
  return (
    <form onSubmit={(e) => onSearchSubmit(e)}>
      <input value={search} onChange={(e) => handleSearchChange(e)}/>
    </form>
  )
}
```

Why:

- Submit on Enter, no separate button needed.
- Renamed props to match App.

5.7 — Created CardPortfolio.tsx

src/Components/Portfolio/CardPortfolio/CardPortfolio.tsx

```
interface Props {
  portfolioValue: string;
  onPortfolioDelete: (e: SyntheticEvent) => void;
}

const CardPortfolio = ({portfolioValue, onPortfolioDelete}: Props) => {
  return (
    <div>
      <h4>{portfolioValue}</h4>
      <form onSubmit={onPortfolioDelete}>
        <input hidden={true} value={portfolioValue} readOnly={true}/>
        <button type="submit">X</button>
      </form>
    </div>
  )
}
```

Why:

- Mirror of AddPortfolio but for deletion. Same hidden-input trick.

5.8 — Created ListPortfolio.tsx

src/Components/Portfolio/ListPortfolio/ListPortfolio.tsx

```
interface Props {
  portfolioValues: string[];
  onPortfolioDelete: (e: SyntheticEvent) => void;
}

const ListPortfolio = ({portfolioValues, onPortfolioDelete}: Props) => {
  return (
    <div>
      <h3>My Portfolio</h3>
```

```
      {portfolioValues && portfolioValues.length > 0 ? (
        portfolioValues.map((portfolioValue) => {
          return (
            <CardPortfolio
              key={portfolioValue}
              portfolioValue={portfolioValue}
              onPortfolioDelete={onPortfolioDelete}
            />
          )
        })
      ) : (
        <h4>No items in portfolio</h4>
      )}
    </div>
  )
}
```

Why:

- Same Card+CardList pattern again. Map over array, render one component per item.

✅ *Search → click Add → appears in portfolio. Click X → removed.*

Feature 6 — React Router + page split

Goal: Multi-page app. URLs map to components.

6.1 — Install router

```
npm install --save react-router
npm install --save react-router-dom
npm install --save @types/react-router
npm install --save @types/react-router-dom
```

6.2 — Created src/Routes/Routes.tsx

src/Routes/Routes.tsx

```
import { createBrowserRouter } from "react-router-dom";
import App from "../App";
import HomePage from "../Pages/HomePage/HomePage";
import SearchPage from "../Pages/SearchPage/SearchPage";

export const router = createBrowserRouter([
  {
    path: "/",
    element: <App />,
    children: [
      { path: "", element: <HomePage /> },
      { path: "search", element: <SearchPage /> },
    ],
  },
]);
```

Why:

- Route tree. `<App />` is the layout shell, children fill in based on URL.
- `path: ""` = the home /. `path: "search"` = /search.

6.3 — Updated index.tsx

src/index.tsx

```
import { RouterProvider } from "react-router-dom";
import { router } from "./Routes/Routes";

root.render(
  <React.StrictMode>
    <RouterProvider router={router} />
  </React.StrictMode>
);
```

Why:

- RouterProvider mounts the router into the React tree.

6.4 — Stripped App.tsx down to layout shell

App.tsx

```
import { Outlet } from 'react-router';
```

```
import './App.css';
import Navbar from './Components/Navbar/Navbar';

function App() {
  return (
    <>
      <Navbar />
      <Outlet />
    </>
  );
}
export default App;
```

Why:

- All search/portfolio logic moved out. App is now just the layout.
- `<Outlet />` = where the matched child route renders.
- `<Navbar />` stays mounted across navigations.

6.5 — Created src/Pages/HomePage/HomePage.tsx

src/Pages/HomePage/HomePage.tsx

```
import Hero from '../../Components/Hero/Hero'

const HomePage = () => {
  return (
    <div>
      <Hero />
    </div>
  )
}
export default HomePage
```

6.6 — Created src/Pages/SearchPage/SearchPage.tsx

Moved all search/portfolio logic here from old App.tsx.

src/Pages/SearchPage/SearchPage.tsx

```
import React, { ChangeEvent, SyntheticEvent, useState } from 'react'
import { CompanySearch } from '../../company';
import { searchCompanies } from '../../api';
import Search from '../../Components/Search/Search';
import ListPortfolio from '../../Components/Portfolio/ListPortfolio/ListPortfolio';
import CardList from '../../Components/CardList/CardList';

const SearchPage = () => {
  const [search, setSearch] = useState<string>("");
  const [portfolioValues, setPortfolioValues] = useState<string[]>([]);
  const [searchResult, setSearchResult] = useState<CompanySearch[]>([]);
  const [serverError, setServerError] = useState<string>("");

  // ...same handlers as before, just relocated

  return (
    <div className="App">
      <Search onSearchSubmit={onSearchSubmit} search={search}
handleSearchChange={handleSearchChange}/>
```

```
      <ListPortfolio portfolioValues={portfolioValues} onPortfolioDelete={onPortfolioDelete}/>
      <CardList searchResults={searchResult} onPortfolioCreate={onPortfolioCreate}/>
      {serverError && <h1>{serverError}</h1>}
    </div>
  )
}
export default SearchPage
```

Why:

- Pages own state and data fetching. Components stay presentational and reusable.

6.7 — Logo redirects to home

In Navbar:

```
<Link to="/">
  <img src={logo} alt="" />
</Link>
```

Why:

- `<Link>` = client-side nav, no full reload, state preserved. `<a href>` = full reload, state wiped.

✅ */ shows HomePage. /search shows SearchPage. Logo click → home.*

Feature 7 — CompanyPage + URL params + useEffect

Goal: Clicking a stock opens its detail page, fetched from API.

7.1 — Added route to Routes.tsx

```
{ path: "company/:ticker", element: <CompanyPage /> },
```

Why:

- `:ticker` is a URL param. /company/AAPL → `ticker = "AAPL"`.

7.2 — Added getCompanyProfile to api.tsx

src/api.tsx

```
export const getCompanyProfile = async (query: string) => {
  try {
    const data = await axios.get<CompanyProfile[]>(

`https://financialmodelingprep.com/stable/profile?symbol=${query}&apikey=${process.env.REACT_APP_API_KEY}`
    )
    return data;
  }
  catch (error: any) {
    console.log("error message from API: ", error.message);
  }
};
```

Why:

- Same try/catch pattern as `searchCompanies`. New endpoint for profile data.

7.3 — Added CompanyProfile interface to company.d.ts

(Big interface with all the company fields — symbol, price, mktCap, sector, etc.)

7.4 — Created src/Pages/CompanyPage/CompanyPage.tsx

src/Pages/CompanyPage/CompanyPage.tsx

```
import React, { useEffect, useState } from 'react'
import { useParams } from 'react-router'
import { CompanyProfile } from '../../company';
import { getCompanyProfile } from '../../api';

const CompanyPage = () => {
  let { ticker } = useParams();
  const [company, setCompany] = useState<CompanyProfile>();

  useEffect(() => {
    const getProfileInit = async () => {
      const result = await getCompanyProfile(ticker!);
      setCompany(result?.data[0]);
    }
    getProfileInit();
  }, []);

  return (
    <>
```

```
      {company ? (
        <div>{company.companyName}</div>
      ) : (
        <div>Company not found!</div>
      )}
    </>
  )
}
export default CompanyPage
```

Why:

- `useParams()` reads URL params. `ticker` is `string | undefined`.
- `useState<CompanyProfile>()` — typed state, undefined initially.
- **`useEffect`** — three patterns:
- • No deps → every render
- • `[]` → mount only
- • `[ticker]` → when ticker changes
- Uses `[]` here (mount-only). Should be `[ticker]` ideally — known issue.
- Async function defined inside the effect, then called. The effect callback itself can't be async (must return nothing or a cleanup function, not a Promise).
- `ticker!` — non-null assertion. "Trust me, exists." Route guarantees it.
- Conditional render handles "still loading" + "not found" states.

✅ *Navigate to /company/AAPL, profile loads.*

Feature 8 — Sidebar + nested routes + CompanyDashboard + Tile

Goal: Inside CompanyPage, navigate between Profile / Income Statement views.

8.1 — Install react-icons

```
npm install react-icons
```

8.2 — Updated Routes.tsx with nested children

```
{
  path: "company/:ticker", element: <CompanyPage />,
  children: [
    { path: "company-profile", element: <CompanyProfile /> },
    { path: "income-statement", element: <IncomeStatement /> },
  ],
},
```

Why:

- Nested children. `/company/AAPL/company-profile` matches both parent and child.

8.3 — Created Sidebar.tsx

src/Components/Sidebar/Sidebar.tsx

```
import { Link } from 'react-router-dom'
import { FaHome } from "react-icons/fa"

const Sidebar = () => {
  return (
    <nav className="...">
      <Link to="company-profile">
        <FaHome /> <h6>Company Profile</h6>
      </Link>
      <Link to="income-statement">
        <FaHome /> <h6>Income Statement</h6>
      </Link>
    </nav>
  )
}
```

Why:

- `to="company-profile"` is a relative path — joins with parent route to give /company/:ticker/company-profile.
- Icons via react-icons.

8.4 — Created CompanyDashboard.tsx

src/Components/CompanyDashboard/CompanyDashboard.tsx

```
import { Outlet } from 'react-router-dom'

type Props = { children: React.ReactNode; }
```

```
const CompanyDashboard = ({children}: Props) => {
  return (
    <div className="...">
      <div className="flex flex-wrap">{children}</div>
      <div className="flex flex-wrap"><Outlet /></div>
    </div>
  )
}
```

Why:

- Layout wrapper that uses both `{children}` and `<Outlet />`.
- `{children}` = content the parent passed in JSX (Tiles).
- `<Outlet />` = content the router decides based on URL (CompanyProfile / IncomeStatement).
- Different sources, both rendered in the same layout.

8.5 — Created Tile.tsx

src/Components/Tile/Tile.tsx

```
type Props = { title: string; subTitle: string; }

const Tile = ({title, subTitle}: Props) => {
  return (
    <div className="...">
      <h5>{title}</h5>
      <span>{subTitle}</span>
    </div>
  )
}
```

Why:

- Reusable card showing title + subtitle. Used for company stats.

8.6 — Created placeholder CompanyProfile.tsx and IncomeStatement.tsx

```
const CompanyProfile = () => <div>CompanyProfile</div>
const IncomeStatement = () => <div>IncomeStatement</div>
```

8.7 — Updated CompanyPage.tsx to use Sidebar + Dashboard + Tile

```
return (
  <>
    {company ? (
      <div className="w-full relative flex ...">
        <Sidebar />
        <CompanyDashboard>
          <Tile title="Company Name" subTitle={company.companyName}/>
        </CompanyDashboard>
      </div>
    ) : (
      <div>Company not found!</div>
    )}
  </>
)
```

✅ *Sidebar links navigate inside CompanyPage. Outlet swaps without remounting parent.*

Feature 9 — Generic Table

Goal: Reusable table that any data + config combo can drive.

9.1 — Created testData.tsx

Mock income statement data for development.

9.2 — Created Table.tsx (v1 — hardcoded data + config)

src/Components/Table/Table.tsx

```
import { testIncomeStatementData } from './testData'

const data = testIncomeStatementData;
type Company = (typeof data)[0];

const configs = [
  { label: "Year", render: (company: Company) => company.acceptedDate },
  { label: "Cost of Revenue", render: (company: Company) => company.costOfRevenue }
]

const Table = () => {
  const renderedRows = data.map((company) => (
    <tr key={company.cik}>
      {configs.map((val: any) => (
        <td>{val.render(company)}</td>
      ))}
    </tr>
  ));
  const renderedHeaders = configs.map((config: any) => (
    <th key={config.label}>{config.label}</th>
  ));
  return (
    <table>
      <thead>{renderedHeaders}</thead>
      <tbody>{renderedRows}</tbody>
    </table>
  );
};
```

Why:

- `type Company = (typeof data)[0]` — derive type from data, no separate interface needed.
- Configs array of `{label, render}` — separate WHAT to render from HOW.
- Two `.maps`: one over data for rows, one over configs for headers.
- Adding a column = push one object to configs. No JSX changes.

9.3 — Created DesignPage.tsx to test Table in isolation

```
const DesignPage = () => (
  <>
    <h1>FinShark Design Page</h1>
    <Table />
  </>
)
```

9.4 — Added route in Routes.tsx

```
{ path: "design-guide", element: <DesignPage /> },
```

✅ */design-guide shows the table with mock data.*

Feature 10 — Outlet context + RatioList + the page recipe (Day 5)

Goal: Pass ticker from CompanyPage down to nested routes. Build CompanyProfile (single object → RatioList) and IncomeStatement (array → Table). Generalise Table.

10.1 — Generalised Table.tsx to accept config + data as props

src/Components/Table/Table.tsx

```
type Props = { config: any; data: any; }

const Table = ({config, data}: Props) => {
  const renderedRows = data.map((company: any) => (
    <tr key={company.cik}>
      {config.map((val: any) => (
        <td>{val.render(company)}</td>
      ))}
    </tr>
  ));
  // ...same headers logic
}
```

Why:

- Was hardcoded to test data. Now reusable.

10.2 — Created RatioList.tsx

src/Components/RatioList/RatioList.tsx

```
type Props = { config: any; data: any; }

const RatioList = ({config, data}: Props) => {
  const renderedRows = config.map((row: any) => (
    <li>
      <p>{row.label}</p>
      <p>{row.subTitle && row.subTitle}</p>
      <div>{row.render(data)}</div>
    </li>
  ))
  return (
    <ul>{renderedRows}</ul>
  )
}
```

Why:

- Like Table but for a single object. One `.map` over config, every row reads from the same data.
- Table → 2D (`data.map → config.map`). RatioList → 1D (just `config.map`).
- Use Table when data is an array. Use RatioList when data is one object.

10.3 — Updated CompanyDashboard.tsx — passes ticker via Outlet context

src/Components/CompanyDashboard/CompanyDashboard.tsx

```
type Props = {
  children: React.ReactNode;
```

```
  ticker: string;  // NEW
}

const CompanyDashboard = ({children, ticker}: Props) => {
  return (
    <div>
      <div>{children}</div>
      <div><Outlet context={ticker}/></div>  {/* NEW */}
    </div>
  )
}
```

Why:

- Nested routes are rendered by the router, not parent JSX → can't pass props directly.
- `<Outlet context={x} />` shares a value with all nested children. No prop drilling.

10.4 — Updated CompanyPage.tsx to forward ticker

```
<CompanyDashboard ticker={ticker!}>
  <Tile title="Company Name" subTitle={company.companyName}/>
</CompanyDashboard>
```

10.5 — Added getKeyMetrics and getIncomeStatement to api.tsx

Same try/catch pattern, new endpoints.

10.6 — Added CompanyKeyMetrics and CompanyIncomeStatement to company.d.ts

10.7 — Built real CompanyProfile.tsx

src/Components/CompanyProfile/CompanyProfile.tsx

```
const tableConfig = [
  {
    label: "Market Cap",
    render: (company: CompanyKeyMetrics) => company.marketCap,
    subTitle: "Total value of all a company's shares of stock",
  },
  // ... 9 more entries
];

const CompanyProfile = () => {
  const ticker = useOutletContext<string>();
  const [companyData, setCompanyData] = useState<CompanyKeyMetrics>();

  useEffect(() => {
    const getCompanyKeyMetrics = async () => {
      const value = await getKeyMetrics(ticker);
      setCompanyData(value?.data[0]);
    };
    getCompanyKeyMetrics();
  }, []);

  return (
    <>
      {companyData ? (
        <RatioList data={companyData} config={tableConfig} />
      ) : (
```

```
        <>Loading...</>
      )}
    </>
  )
}
```

Why:

- `useOutletContext<string>()` reads ticker from parent's Outlet context.
- Page recipe: read ticker → useState for data → useEffect mount fetch → conditional render → display component.
- Single object data → RatioList.

10.8 — Built real IncomeStatement.tsx

Same recipe but data is an array → uses Table.

src/Components/IncomeStatement/IncomeStatement.tsx

```
const IncomeStatement = () => {
  const ticker = useOutletContext<string>();
  const [incomeStatement, setIncomeStatement] = useState<CompanyIncomeStatement[]>();

  useEffect(() => {
    const incomeStatementFetch = async () => {
      const result = await getIncomeStatement(ticker);
      setIncomeStatement(result!.data);
    };
    incomeStatementFetch();
  }, [])

  return (
    <>
      {incomeStatement ? (
        <>
        <Table config={configs} data={incomeStatement}/>
        </>
      ) : (
        <>Loading...</>
      )}
    </>
  )
}
```

Why:

- Same architecture, different data shape and display component.

✅ *Click sidebar links → fetches and shows real data.*

Feature 11 — Balance Sheet (Day 6)

Goal: Third financial statement. Same recipe, third application.

11.1 — Added CompanyBalanceSheet interface to company.d.ts

11.2 — Added getBalanceSheet to api.tsx

11.3 — Added route in Routes.tsx

```
{ path: "balance-sheet", element: <BalanceSheet /> }
```

11.4 — Added link in Sidebar.tsx

11.5 — Created BalanceSheet.tsx — same recipe as CompanyProfile

(single object → RatioList)

src/Components/BalanceSheet/BalanceSheet.tsx

```
const config = [
  {
    label: <div className="font-bold">Total Assets</div>,  // JSX as label
    render: (c: CompanyBalanceSheet) => c.totalAssets,
  },
  // ...
];

const BalanceSheet = () => {
  const ticker = useOutletContext<string>();
  const [balanceSheet, setBalanceSheet] = useState<CompanyBalanceSheet>();

  useEffect(() => {
    const getData = async () => {
      const value = await getBalanceSheet(ticker!);
      setBalanceSheet(value?.data[0]);
    };
    getData();
  }, []);

  return (
    <>
      {balanceSheet ? <RatioList config={config} data={balanceSheet} /> : <h1>Not found</h1>}
    </>
  )
}
```

Why notable

- First time `label` is JSX (`<div className="font-bold">`) instead of a string.
- Works because `{row.label}` accepts any ReactNode, not just strings.

✅ *Third tab works. Recipe holds.*

Feature 12 — CashFlow + Spinner + CompFinder + direct landing (Day 7)

12.1 — Install react-spinners

```
npm install react-spinners
```

12.2 — Created Spinner.tsx

src/Components/Spinner/Spinner.tsx

```
import { ClipLoader } from 'react-spinners';

type Props = { isLoading?: boolean; }

const Spinner = ({isLoading = true}: Props) => (
  <div id="loading-spinner">
    <ClipLoader color='#36d7b7' loading={isLoading} size={35}/>
  </div>
)
```

Why:

- Wrapper around the third-party component. One place to swap libraries later.
- `isLoading?: boolean` with `= true` — optional prop with default.

12.3 — Replaced "Loading..." text with <Spinner /> everywhere

CompanyPage, CompanyProfile, IncomeStatement, BalanceSheet.

12.4 — Added getCashFlowStatement and CompanyCashFlow interface

12.5 — Created CashFlowStatement.tsx

Same recipe again (array → Table).

12.6 — Added route + Sidebar link

12.7 — Added getCompData to api.tsx (peer companies)

12.8 — Created CompFinder.tsx

src/Components/CompFinder/CompFinder.tsx

```
type Props = { ticker: string; }

const CompFinder = ({ticker}: Props) => {
  const [companyData, setCompanyData] = useState<CompanyCompData[]>([]);

  useEffect(() => {
    const getComps = async () => {
      const value = await getCompData(ticker!);
      if (value?.data) setCompanyData(value.data);
    };
    getComps();
  }, [ticker]);  // NOTE: real dep array
```

```
  return (
    <div>
      {companyData.map((c) => <CompFinderItem ticker={c.symbol} />)}
    </div>
  )
}
```

Why:

- Receives `ticker` as a prop, not from Outlet context — it's a child component, not a route.
- useEffect deps `[ticker]` (correct). Pages used `[]` (kind of bug — workaround comes next).

12.9 — Created CompFinderItem.tsx

`src/Components/CompFinder/CompFinderItem/CompFinderItem.tsx`

```
import { Link } from 'react-router-dom';

const CompFinderItem = ({ticker}: Props) => (
  <Link reloadDocument to={`/company/${ticker}/company-profile`}>
    {ticker}
  </Link>
)
```

Why:

- `reloadDocument` — full page reload on click. Forces all the useEffects with [] deps to refetch for the new ticker.
- It's a workaround. Proper fix would be `[ticker]` deps everywhere.

12.10 — Added CompFinder to CompanyPage + direct landing

```
<CompanyDashboard ticker={ticker!}>
  <Tile title="Company Name" subTitle={company.companyName}/>
  <Tile title="Price" subTitle={company.price.toString()}/>
  <Tile title="Sector" subTitle={company.sector}/>
  <CompFinder ticker={company.symbol} />
</CompanyDashboard>
```

✅ *Loading spinner everywhere. Peer companies clickable. Click peer → opens their profile.*

Feature 13 — Number formatting + 10-K + complete frontend (Day 8)

13.1 — Created src/Helpers/NumberFormatting.tsx

src/Helpers/NumberFormatting.tsx

```
export const formatLargeMonetaryNumber: any = (number: number) => {
  if (number < 0) return "-" + formatLargeMonetaryNumber(-1 * number);
  if (number < 1000) return "$" + number;
  if (number < 1_000_000) return "$" + (number / 1000).toFixed(1) + "K";
  if (number < 1_000_000_000) return "$" + (number / 1_000_000).toFixed(1) + "M";
  if (number < 1_000_000_000_000) return "$" + (number / 1_000_000_000).toFixed(1) + "B";
  if (number < 1_000_000_000_000_000) return "$" + (number / 1_000_000_000_000).toFixed(1) + "T";
};

export const formatLargeNonMonetaryNumber: any = (number: number) => {
  // same shape, no $ prefix
};

export const formatRatio = (ratio: number) => {
  return (Math.round(ratio * 100) / 100).toFixed(2);
};
```

Why:

- Pure utility functions. No JSX, no React.
- Used in 4 places — define once, import everywhere. Single source of truth.
- `toFixed(1)` returns a string.

13.2 — Wired formatters into all configs

Each config's `render` now passes the value through a formatter:

```
{ label: "Market Cap", render: (c: CompanyKeyMetrics) =>
formatLargeNonMonetaryNumber(c.marketCap), subTitle: "..." }
```

Updated CompanyProfile, BalanceSheet, IncomeStatement, CashFlowStatement.

13.3 — Switched 10-K data source

FMP's 10-K endpoint is premium → switched to massive.com's filings API.

13.4 — Updated CompanyTenK interface in company.d.ts

New field names matching Massive's schema: `ticker, filing_date, period_end, text, section, filing_url`.

13.5 — Added getTenK to api.tsx

src/api.tsx

```
export const getTenK = async (query: string) => {
  try {
    const data = await axios.get<{ results: CompanyTenK[] }>(  // NOTE: wrapped in {results: ...}
      `https://api.massive.com/stocks/filings/10-
K/vX/sections?ticker=${query}&section=business&limit=10&sort=period_end.desc&apiKey=${process.env.REACT_APP_MAS
    )
    return data;
```

```
  }
  catch (error: any) {
    console.log("error message from API: ", error.message);
  }
};
```

Why:

- New API wraps response in `{ results: [...] }` instead of returning bare array.
- Generic must match: `<{ results: CompanyTenK[] }>`.
- Reading the response: `value?.data?.results` (not `value?.data`).

13.6 — Created TenKFinder.tsx — fetches and maps filings

src/Components/TenKFinder/TenKFinder.tsx

```
const TenKFinder = ({ticker}: Props) => {
  const [companyData, setCompanyData] = useState<CompanyTenK[]>([]);

  useEffect(() => {
    const getTenKData = async () => {
      const value = await getTenK(ticker);
      setCompanyData(value?.data?.results!);  // NOTE: results not data
    };
    getTenKData();
  }, []);

  return (
    <div>
      {companyData.map((tenK) => <TenKFinderItem tenK={tenK} />)}
    </div>
  )
}
```

13.7 — Created TenKFinderItem.tsx with URL rewriting

src/Components/TenKFinder/TenKFinderItem/TenKFinderItem.tsx

```
const TenKFinderItem = ({tenK}: Props) => {
  const htmlUrl = tenK.filing_url.replace(
    /\/([\d-]+)\.txt$/,
    (_, accession) =>
      `/${accession.replace(/-/g, '')}/${tenK.ticker.toLowerCase()}-${tenK.period_end.replace(/-
/g, '')}.htm`
  );

  return (
    <a href={htmlUrl}>
      10-K {tenK.period_end}
    </a>
  )
}
```

Why:

- Massive returns a `.txt` filing URL. Original SEC HTML version is more readable.
- `replace(/regex/, callback)` — regex with a callback. Callback receives `(fullMatch, group1)`. `_` = "don't care about full match." `accession` = the captured group.

- Builds the SEC HTML URL by stripping dashes from the accession number and substituting the ticker + period.

13.8 — Added TenKFinder + Market Cap tile + description to CompanyPage

```
<CompanyDashboard ticker={ticker!}>
  <Tile title="Company Name" subTitle={company.companyName}/>
  <Tile title="Price" subTitle={"$" + company.price.toString()}/>
  <Tile title="Market Cap" subTitle={"$" + company.marketCap.toString()} />
  <Tile title="Sector" subTitle={company.sector}/>
  <CompFinder ticker={company.symbol} />
  <TenKFinder ticker={company.symbol} />
  <p>{company.description}</p>
</CompanyDashboard>
```

🏁 Frontend complete.

Summary — every file in final form

```
src/
├── Components/
│   ├── Card/Card.tsx
│   ├── CardList/CardList.tsx
│   ├── Search/Search.tsx
│   ├── Navbar/Navbar.tsx
│   ├── Hero/Hero.tsx
│   ├── Sidebar/Sidebar.tsx
│   ├── CompanyDashboard/CompanyDashboard.tsx
│   ├── CompanyProfile/CompanyProfile.tsx
│   ├── IncomeStatement/IncomeStatement.tsx
│   ├── BalanceSheet/BalanceSheet.tsx
│   ├── CashFlowStatement/CashFlowStatement.tsx
│   ├── Spinner/Spinner.tsx
│   ├── Tile/Tile.tsx
│   ├── Table/
│   │   ├── Table.tsx
│   │   └── testData.tsx
│   ├── RatioList/RatioList.tsx
│   ├── CompFinder/
│   │   ├── CompFinder.tsx
│   │   └── CompFinderItem/CompFinderItem.tsx
│   ├── TenKFinder/
│   │   ├── TenKFinder.tsx
│   │   └── TenKFinderItem/TenKFinderItem.tsx
│   └── Portfolio/
│       ├── AddPortfolio/AddPortfolio.tsx
│       ├── CardPortfolio/CardPortfolio.tsx
│       └── ListPortfolio/ListPortfolio.tsx
├── Pages/
│   ├── HomePage/HomePage.tsx
│   ├── SearchPage/SearchPage.tsx
│   ├── CompanyPage/CompanyPage.tsx
│   └── DesignGuide/DesignGuide.tsx
├── Routes/Routes.tsx
├── Helpers/NumberFormatting.tsx
├── App.tsx
├── api.tsx
├── company.d.ts
└── index.tsx
```